

Transactions Briefs

Stochastic Computing Using Amplitude and Frequency Encoding

Yuhao Chen^{ID} and Hongge Li^{ID}

Abstract—Stochastic computing (SC) is a type of logic computation based on stochastic bit stream instead of the binary numbers (BNs). It is pseudo-analog computation in the digital domain that can realize multiplication, addition, and complex function computations. In this brief, a new encoding method, referred to as amplitude and frequency encoding (AFE) for SC, is presented. It extends SC to use multibit streams instead of bit streams to represent data. This method still uses the expectation of streams to achieve computation. Compared with the conventional stochastic bit stream, AFE realizes low-latency and low-area occupation than the conventional SC method. The rationality of the circuits is also examined from a mathematical perspective. The hardware logic circuit of AFE SC, including the multiplier, adder, and converter, was designed and implemented by a field-programmable gate array (FPGA). In addition, the latency of the computing, area, and precision of AFE SC was measured based on the FPGA board.

Index Terms—Amplitude and frequency encoding (AFE), low latency, stochastic computing (SC).

I. INTRODUCTION

Stochastic computing (SC), originally advocated by Gaines [1] in 1963, is an important method to realize pulse computation [2]. In the past few years, SC has become popular again because of its extremely low power consumption and small hardware occupation of the computation core. It has been applied to a wide variety of applications, such as image processing [3]–[6], neural networks [7], [8], and deep learning [9]–[11].

In the SC paradigm, digital logical computation is performed on stochastic bit streams instead of a binary number (BN). The stochastic bit stream is represented by the probability of firing “1” based on the stream length of the accumulation time.

This particular numerical representation has a great advantage in the use of hardware resources in processing complex functions and polynomial operations. Since data in circuits are usually recorded as binary representations, the first step of SC is to use a BN to stochastic bit stream (SB) converter to obtain SC streams. However, the BN-SB converter also increases resource consumption remarkably [12], [13]. Because of the problems of SC, such as high latency, poor precision, and nonexcellent energy efficiency, some improvements in computing rules have been made for conventional frequency encoding, including bipolar or unipolar representation.

Sim and Lee [10] proposed a new computing rule to obtain better latency. Liu and Han [14]–[16] used Sobol to optimize precision. Jenson and Riedel [17]–[19] proposed a deterministic method to achieve accurate calculation. Parhi [20] proposed hybrid unipolar and bipolar formats to achieve better energy efficiency, and Lee *et al.* [21]

developed a digital–analog hybrid method, which is more energy efficient than the binary logic method.

However, some problems are caused by using the encoding method instead of computing rules, such as high latency. The main problem of the bit stream is its efficiency in carrying information. When the stream length is N , it can only represent N different data, which is weaker than binary representation (n -bit can represent 2^n different data).

Based on the above research results and the remaining challenges, AFE design ideas and methods are proposed. The main contributions of this study are as follows.

- 1) A new encoding method, amplitude and frequency encoding (AFE), for SC is proposed for latency and hardware occupation.
- 2) The circuit of the multiplier, adder, and BN-SB and SB-BN converter for AFE was designed.
- 3) The circuit of the quadratic polynomial based on the AFE was tested, and the latency decreased exponentially.

The remainder of this brief is organized as follows. In Section II, the existing SC method is described, the new encoding method, AFE SC, is discussed, and the design of the AFE SC circuit is described. In Section III, the error of SC is discussed, and the precision and area efficiency of conventional and AFE SC are compared. Section IV concludes this brief.

II. STOCHASTIC COMPUTING (SC)

A. Existing SC Methods

The classic SC [1] for a quadratic polynomial calculation is shown in Fig. 1(a). BN-SB is composed of a random number generator (RNG) and comparator. Through the BN-SB converter, A and B are converted into stochastic streams S_A and S_B . The probability of two streams being “1” is $P(S_A) = A/2^n$ and $P(S_B) = B/2^m$. After passing through the “AND” gate, a new stochastic stream $S_{A,B}$ is obtained. The probability of the output stream being “1” is $P(S_{A,B}) = P(S_A)P(S_B) = AB/2^{n+m}$, which realizes the multiplication of two input streams S_A and S_B . The counter accumulates four stochastic streams, $S_{A1,B1} - S_{A4,B4}$, to obtain the full result of the polynomial.

Thus, the SC results may be inaccurate for the following reasons. First, two input streams should be unrelated; otherwise, $P(S_{A,B}) = P(S_A)P(S_B)$ is false. Second, the stochastic stream is essentially a result of the N_F -fold Bernoulli experiment, and the number of “1” contained in the stream satisfies the binomial distribution [22]. The length of an SB increases exponentially with the desired resolution. For example, to obtain the probability of a stream with 8-bit resolution (2^{-8}), one must observe at least 2^{16} bits of stream length using classic SC [18].

Some optimization methods for precision and latency based on the classical method have been presented. The method in Fig. 1(b) is the optimization for latency, which can achieve a 50% average latency reduction [10]. The method in Fig. 1(c) is aimed at optimizing the precision. Using the periodicity of the pseudorandom sequence, zero error calculation can be realized [17]. To obtain the probability of a stream with $(n + m)$ -bit resolution, only 2^{n+m} bits are required.

Manuscript received November 18, 2021; revised January 10, 2022; accepted February 4, 2022. Date of publication February 25, 2022; date of current version April 26, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant 62071019. (Corresponding author: Hongge Li.)

The authors are with the School of Electronic Information Engineering, Beihang University, Beijing 100191, China (e-mail: zy1902608@buaa.edu.cn; honggeli@buaa.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TVLSI.2022.3150569>.

Digital Object Identifier 10.1109/TVLSI.2022.3150569

1063-8210 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

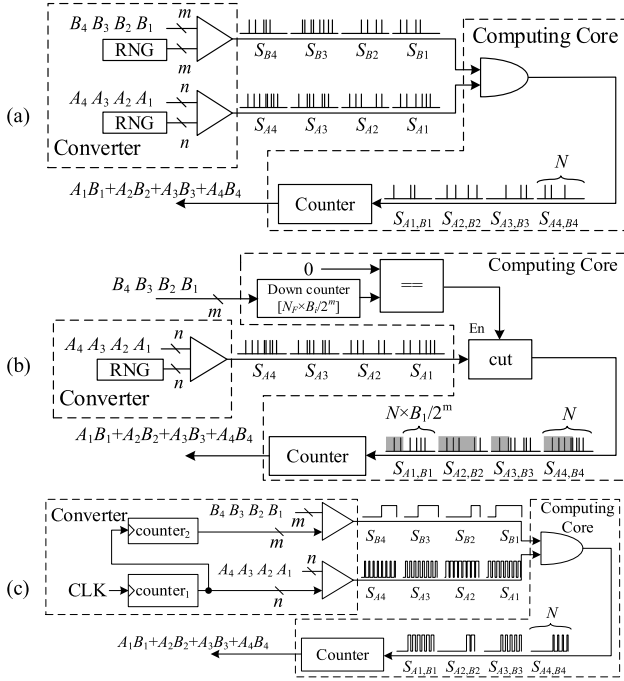


Fig. 1. Frequency encoded stochastic computing: (a) classic method [1], (b) BISC [10], and (c) deterministic method [17] (A and B are unsigned binary number with n -bit and m -bit, respectively).

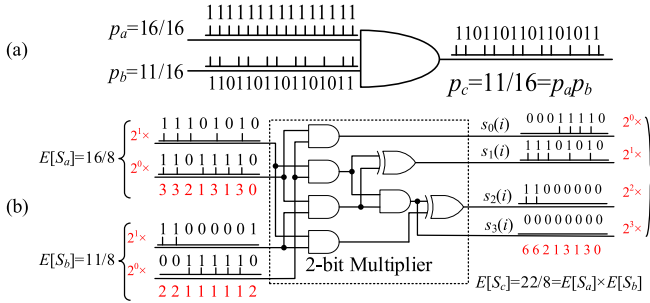


Fig. 2. Multiplication of AFE SC. (a) Bitstream. (b) Multibit streams with two bits.

B. AFE SC

In previous studies, SC was usually considered to use the probability of “1” in a stream to represent a value. $x(1), x(2), \dots, x(N)$ is used to represent the stream. Every $x(i)$ in the bit stream is generated by a discrete random variable X with a probability distribution of $P(X = 1) = p$, $P(X = 0) = 1 - p$, where p is the probability value. According to the definition of expectation in a previous report [22], the expectation of X can be obtained as follows:

$$E[X] = 1 \cdot p + 0 \cdot (1 - p) = p. \quad (1)$$

According to (1), it can also be regarded that the SC uses the discrete random variable X as a signal source to generate a stochastic stream, and $E[X]$ represents the value of the stream. Since $X = 1$ or 0 in conventional SC, the stochastic streams are 1-bit streams.

If SC is understood as described above, the discrete random variable should have a larger range instead of being limited to 1 or 0. Therefore, SC can be extended to use multibit streams instead of 1-bit streams. Examples of SC with a 2-bit stream are shown in Figs. 2 and 3.

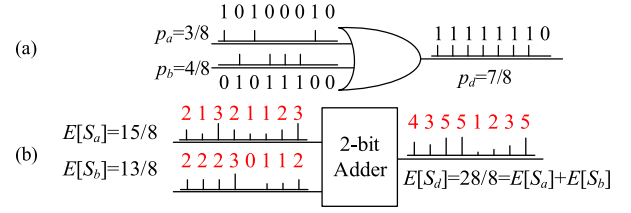


Fig. 3. Addition of AFE SC. (a) Bitstream. (b) Multibit streams with two bits.

According to the research [22], the mathematical characters of expectation are shown in the following equations, where C is a constant, X, Y are discrete random variables, and X, Y are independent in (4)

$$E[C] = C \quad (2)$$

$$E[C \cdot X] = C \cdot E[X] \quad (3)$$

$$E[X \cdot Y] = E[X]E[Y] \quad (4)$$

$$E[X + Y] = E[X] + E[Y]. \quad (5)$$

1) *Multiplication and Addition*: If multibit streams (non-1-bit streams), $s_a(i)$ and $s_b(i)$, are multiplied number by number, a new multibit stream $s_c(i)$ can be obtained

$$s_c(i) = s_a(i)s_b(i). \quad (6)$$

According to (4), if $s_a(i)$ and $s_b(i)$ are independent, the expectation of $s_c(i)$ is the product of $E[S_a]$ and $E[S_b]$, as shown in the following equation. Therefore, the multiplication of multibit streams can be realized by multiplying two numbers together at the same location of the two streams

$$E[S_c] = E[S_a S_b] = E[S_a]E[S_b]. \quad (7)$$

By adding two multibit streams, $s_a(i)$ and $s_b(i)$, number by number, a new multibit stream $s_d(i)$ can be obtained

$$s_d(i) = s_a(i) + s_b(i). \quad (8)$$

According to (5), the expectation of $s_d(i)$ is equal to $E[S_a] + E[S_b]$, as shown in the following equation. Therefore, the addition of multibit stream can be realized by adding two numbers at the same location of the two streams

$$E[S_d] = E[S_a + S_b] = E[S_a] + E[S_b]. \quad (9)$$

Using Fig. 2(b) to explain the details, BNs 16 and 11 should first be scaled into $[0, 2^2 - 1]$ to satisfy the representation range of the stream with two bits. Therefore, 2-bit streams are encoded with $E[S_a] = 16/8$ and $E[S_b] = 11/8$ to represent 16 and 11, respectively. Then, using this new encoding method for SC, the 5-bit multiplication is realized by a 2-bit multiplier, as shown in Fig. 2(b). And the 5-bit addition is also realized by the adder of AFE with two bits stream, as shown in Fig. 3(b).

Through this new SC method with multibit streams, the AFE SC architecture can trade off latency and hardware occupation, which is further explained in Section II-B2. Based on the proposed method, the information is encoded on both the amplitude and the frequency of the multibit stream.

2) *BN-SB Converter*: Two types of BN-SB circuits, Type-A and Type-B, are proposed, as shown in Fig. 4. They both generate unipolar stream, which use an extra signed bit to realize signed computing. BN-SB (Type-A) uses one BN x (n -bit) as input to realize the encoding of multibit stream. An example of the encoding process for 4-bit stochastic stream is taken in Fig. 4(a), and x_i represents the i th bit of x .

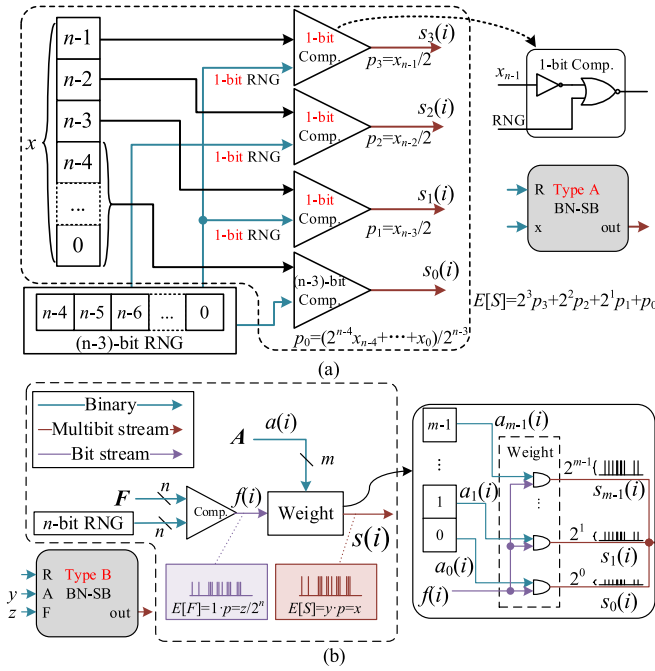


Fig. 4. Architecture of BN-SB converter for AFE SC. (a) Type-A, use single binary number to encode stream. (b) Type-B, use two binary numbers to encode stream.

The low \$(n-3)\$-bit of \$x\$ is compared with a \$(n-3)\$-bit RNG to obtain the bitstream \$s_0(i)\$. High 3-bit of \$x\$ is compared with three 1-bit out of RNGs, respectively, to obtain \$s_1(i)\$, \$s_2(i)\$, and \$s_3(i)\$. Because bit streams in the same type-A AFE will not be multiplied with each other, independence is not necessary. Thus, the 1-bit RNGs can be gained from arbitrary bit of the \$(n-3)\$-bit RNG without extra hardware occupation. The 4-bit stream output versus input \$x\$ has the following relationship:

$$E[S_0] = p_0 = (2^{n-4}x_{n-4} + \dots + 2^1x_1 + x_0)/2^{n-3} \quad (10)$$

$$E[S_i] = p_i = x_{n-4+i}/2, \quad i = 1, 2, 3. \quad (11)$$

Thus, the expectation of the 4-bit stream output by BN-SB (Type-A) is proportional to \$x\$, which realizes the multibit stream encoding for BN \$x\$

$$\begin{aligned} E[S] &= 2^3 p_3 + 2^2 p_2 + 2^1 p_1 + 2^0 p_0 \\ &= (2^{n-1}x_{n-1} + 2^{n-2}x_{n-2} + \dots + x_0)/2^{n-3} = x/2^{n-3}. \end{aligned} \quad (12)$$

Type-B illustrated in Fig. 4(b) is another method for encoding multibit stream, and it uses two BNs to encode multibit stream. The output multibit stream represents the product of the two input numbers. The BN \$x(n+m)\$-bit to be encoded is the product of \$y\$ and \$p\$, \$x = y \cdot p = y \cdot z/2^n\$. \$z\$ is \$n\$-bit, and \$y\$ is \$m\$-bit, \$p \in [0,1)\$. The bit stream \$f(i)\$ representing \$p\$ is generated by the comparison between \$z\$ and random number, which is the same as the BN-SB circuit for conventional SC

$$E[F] = p = z/2^n. \quad (13)$$

Then, if \$a(i) = y\$ and \$i = 1, 2, \dots, N\$, the expectation of \$A\$ is

$$E[A] = y. \quad (14)$$

Finally, \$s(i)\$ is obtained by multiplying \$a(i)\$ and \$f(i)\$ together, as shown in the right side of Fig.4(b). Here, \$a_0(i), a_1(i), \dots, a_{m-1}(i)\$ indicate the \$m\$ bits of \$a(i)\$. The \$m\$ bits of \$s(i)\$ have the same

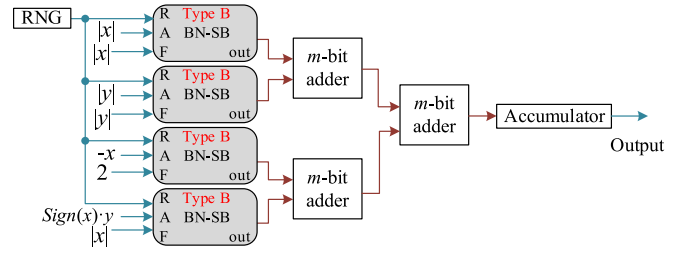


Fig. 5. Use BN-SB (Type-B) as multiplier for the quadratic polynomial fitting: \$z = x^2 + y^2 - 2x + xy\$.

representation as \$a(i)\$

$$s(i) = \sum_{j=0}^{m-1} 2^j s_j(i) = \sum_{j=0}^{m-1} 2^j [a_j(i) \cdot f(i)]. \quad (15)$$

The value expressed by \$s(i)\$ is determined by the expectation of \$s(i)\$. Since \$A\$ is a constant, the expectation is obtained according to (3), as shown in the following equation. Therefore, \$s(i)\$ is the multibit stream representing \$x\$, and the circuit achieves the purpose of converting the BN \$x\$ into a multibit stream

$$E[S] = \left(\sum_{j=0}^{m-1} 2^j y_j \right) E[F] = y \cdot E[F] = y \cdot p = x. \quad (16)$$

In conventional SC, bit streams represent different information by the different frequencies of “1” pulses, while multibit streams use not only the pulse frequency but also the pulse amplitude to represent different information. Therefore, this new SC method with multibit streams is named as “amplitude and frequency encoding.”

AFE has a lower requirement for stream length \$N\$ than conventional SC to reach the same resolution. When converting a \$n\$-bit BN \$x\$ to \$j\$-bit stream, it is sufficient to have \$(n-j)\$-bit resolution (\$N = 2^{n-j}\$) using Type-B for every bitstream. And when using Type-A, \$(n-j+1)\$-bit resolution (\$N = 2^{n-j+1}\$) is needed, where \$n\$-bit resolution (\$N = 2^n\$) is required for conventional SC.

3) *SB-BN Converter*: Similar to conventional SC, the strategy of recovering values from the AFE stream is to estimate the expectation by computing the average of a sufficiently long AFE stream, so accumulators are used to achieve the function of SB-BN

$$\text{mean}[s(i)] = \sum_{i=1}^N s(i)/N \approx E[S]. \quad (17)$$

To avoid a division operation, \$N\$ takes an integer power of 2, which can achieve division by shifting.

4) *Special Multiplier for Quadratic Polynomial*: As the BN-SB circuit (Type-B) completes stream encoding, it also naturally achieves a multiplication operation. Therefore, when realizing the first multiplication of polynomials, BN-SB (Type-B) can be used as a multiplier that uses two binary inputs to output the AFE stream. The advantage of this special usage is more prominent when operating a quadratic polynomial because the quadratic polynomial only has one multiplication in every monomial.

When dealing with the quadratic polynomial shown in Fig. 5, \$z = x^2 + y^2 - 2x + xy\$, where \$x\$ and \$y\$ are \$m\$-bit numbers. Binary operation requires four \$m\$-bit multipliers and three \$2m\$-bit adders, whereas the AFE SC requires only four BN-SB converters and three \$m\$-bit adders.

III. ERROR DISCUSSION FOR AFE SC

A. Probability Distribution of AFE Stream

As previously mentioned, \$s(i)\$ is generated by the discrete random variables \$S\$. The range of \$S\$ can be adjusted according to operation.

If S is m -bit, $S \in \mathbb{Z} \cap [0, L]$ and $L = 2^m - 1$, the probability distribution of S is $P(S = 0) = p_0$, $P(S = 1) = p_1, \dots$, $P(S = L) = p_L$

$$E[S] = p_0 \cdot 0 + p_1 \cdot 1 + \dots + p_L \cdot L. \quad (18)$$

According to the encoding method in Section II-B2, there are two numbers, $x_a = y_a p_a$ and $x_b = y_b p_b$. If the AFE streams representing x_a and x_b are generated by the BN-SB (Type-B) circuit, the probability distribution of S_a is $P(S_a = 0) = 1 - p_a$, $P(S_a = y_a) = p_a$, the probability distribution of S_b is $P(S_b = 0) = 1 - p_b$, $P(S_b = y_b) = p_b$, and the expectations of S_a and S_b are

$$E[S_a] = (1 - p_a) \cdot 0 + y_a p_a = y_a p_a = x_a \quad (19)$$

$$E[S_b] = (1 - p_b) \cdot 0 + y_b p_b = y_b p_b = x_b. \quad (20)$$

If $s_c(i)$ is the multiplication of $s_a(i)$ and $s_b(i)$, $s_a(i)$ and $s_b(i)$ are independent, and the probability distribution of S_c can be obtained, $P(S_c = 0) = 1 - p_a p_b$, $P(S_c = y_a y_b) = p_a p_b$

$$E[S_c] = (1 - p_a p_b) \cdot 0 + (y_a y_b)(p_a p_b) = x_a x_b. \quad (21)$$

If $s_d(i)$ is the addition of $s_a(i)$ and $s_b(i)$, the probability distribution of S_d can be obtained as $P(S_d = 0) = (1 - p_a)(1 - p_b)$, $P(S_d = y_a) = p_a(1 - p_b)$, $P(S_d = y_b) = p_b(1 - p_a)$, and $P(S_d = y_a + y_b) = p_a \cdot p_b$

$$\begin{aligned} E[S_d] &= (1 - p_a)(1 - p_b) \cdot 0 + p_a(1 - p_b)y_a \\ &\quad + p_b(1 - p_a)y_b + p_a p_b(y_a + y_b) \\ &= p_a y_a + p_a y_a = x_a + x_b. \end{aligned} \quad (22)$$

Observing the above computation, the stream output from BN-SB (Type-B) in Fig. 4 can only be 0 or y . After the multiplication operation, S_c still takes two values, which is 0 or $y_a y_b$. After the add operation, S_d takes four values, and the probability distribution of S_d becomes more complicated. If a polynomial operation is performed, the probability distribution and expectation can be obtained according to the above process. Its probability distribution is much more complicated because of the addition operations in the polynomial.

B. Precision Experiment for AFE SC

Because of the existence of progressive precision [23], precision can easily be changed by choosing different stream lengths, which is one of the advantages of SC. Since the hardware implementation of true RNG is more complex, a pseudorandom sequence is usually used in SC. When using a pseudorandom sequence, obtaining progressive precision through statistics becomes a better strategy, and it is used in the following test.

Using binary computation as the benchmark, the deviation in multiplication between SC and the binary method is calculated. The mean absolute error (MAE) is calculated as the index to measure the precision in (23). $SC(A, B, N)$ is the result of SC for $A \times B$ with stream length N , and $P(A, B)$ is the theoretical value for $A \times B$, where A is n -bit unsigned BN and B is m -bit. When $MAE(N) = 0$, the result of SC is completely consistent with the binary calculations, which means that a zero-error calculation is achieved [24]

$$MAE(N) = \frac{\sum_{A=0}^{2^n-1} \sum_{B=0}^{2^m-1} |SC(A, B, N) - P(A, B)|}{2^{n+m}}. \quad (23)$$

Fig. 6 shows the progressive precision of the four methods with different RNGs when performing 4-bit multiplication ($n = m = 4$ in Fig. 1). Among the bit stream methods [1], [10], [17], binary-interfaced SC (BISC) has the best progressive precision. AFE SC has a speed increase 2^m times that of BISC to achieve zero error calculation. As for progressive precision, the AFE SC also has a better performance than BISC method.

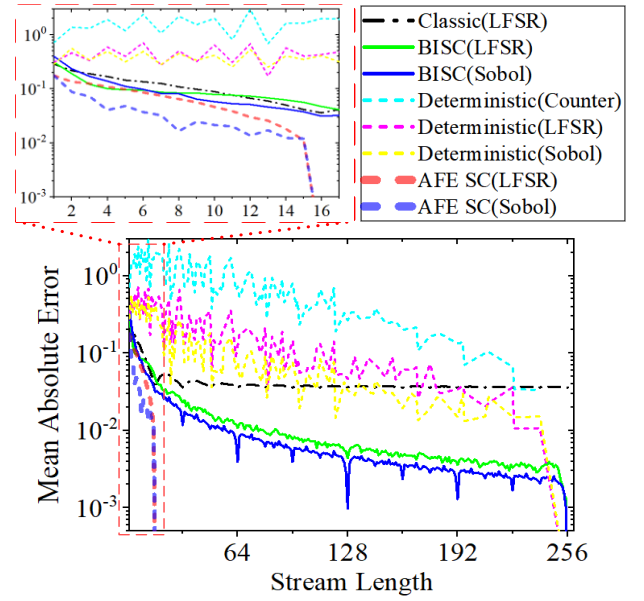


Fig. 6. Progressive precision for classic [1], BISC [10], deterministic [17], and AFE SC for 5-bit multiplication of two input.

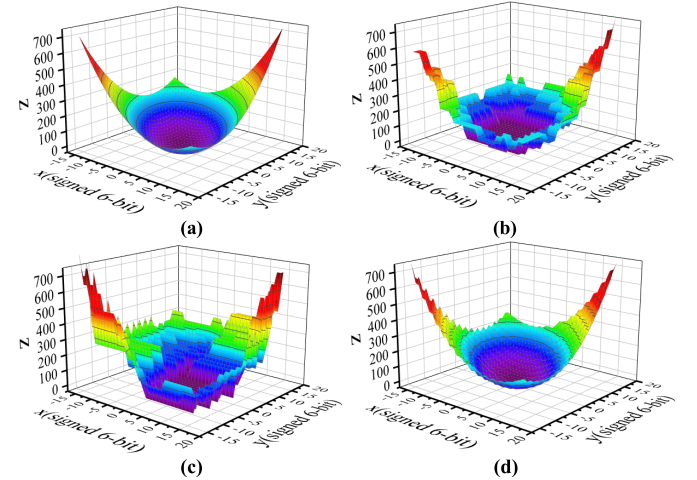


Fig. 7. Quadratic polynomial fitting: $z = x^2 + y^2 - 2x + xy$, where x and y are signed 6-bit binary and stream length $N = 16$: (a) binary, (b) BISC [10], (c) deterministic [17], and (d) AFE.

The AFE SC in the test uses the BN-SB (Type-B) circuit in Fig. 4(b) as a multiplier. The period of n -bit RNG is 2^n , and every number $r \in \mathbb{Z} \cap [0, 2^n - 1]$ appears once in one period of RNG. In the encoding process of Type-B, $a(i)$ is a constant, $a(i) = y$. When $N = 2^n$, the result of comparison satisfied $z > r$ should appear z times. The mean of $f(i)$ should be equal to $z/2^n$. Therefore, the mean of the multibit output should also have zero-error with the product of y and p as shown in (24), which means BN-SB (Type-B) as multiplier produces the accurate results

$$\text{mean}[s(i)] = y \sum_{i=1}^N f(i)/2^n = y \cdot p = x/2^n. \quad (24)$$

A quadratic polynomial fitting was performed using conventional SC and AFE SC (see Fig. 5), and the results are shown in Fig. 7. Under the same stream length, the AFE SC has a clear advantage.

Table I shows the latency of AFE and conventional SCs to reach the same precision, using MAE to measure error. As mentioned in [19], when $N > 2^n$, the generated bit stream repeats and so the precision of classic method [1] has no more improvements after this time.

TABLE I
LATENCIES OF DIFFERENT STOCHASTIC METHODS

Method			Classic [1]	BISC [10]		Deterministic [17]		AFE SC	
RNG			LFSR	LFSR	Sobol	LFSR	Sobol	LFSR	Sobol
Bit width	2 bit	MAE	0.06	0.08	0.06	0.07	0.07	0.06	0.08
		SL	2 ²	2 ²	2 ²	[3×3]	[3×3]	3	3
	4 bit	MAE	0.04	0.04	0.04	0.04	0.04	0.04	0.04
		SL	2 ⁴	17	15	[8×8]	[8×8]	11	5
	8 bit	MAE	0.0083	0.008	0.008	0.008	0.008	0.008	0.008
		SL	2 ⁸	483	122	[143×143]	[30×30]	184	29
	10 bit	MAE	0.0042	0.004	0.004	0.004	0.004	0.004	0.004
		SL	2 ¹⁰	2117	279	[800×800]	[49×49]	732	56

RNG has the same bit width as the input data, SL means the Stream Length.

TABLE II
AREA EFFICIENCIES OF DIFFERENT METHODS WITH 10-BIT DATA

Method	Area (LUT)	Latency (Ck)	Power (mw)	Energy* (nJ)	Area Efficiency**
Classic [1]	24	1,024	7	17.92	40.7
Deterministic [17]	24	2,401	5	30.01	17.4
BISC [10]	36	279	7	4.88	99.6
AFE SC	21	56	6	0.84	850.3
Binary logic	115	1	12	0.03	8695.7

* Energy=Latency×Power/Frequency, Frequency=400 MHz.

** 10%/(LUT×latency)

Therefore, the stream length N in the test is also no more than 2^n . Based on the best precision achieved by classic SC operating on BNs with different bit widths, the stream length of other methods to achieve the same precision is test.

The multiplication circuit for different SC methods with 10-bit unsigned data was implemented by field-programmable gate array (FPGA), as shown in Table II. Latency represents the stream length required to achieve a similar precision ($MAE \approx 0.004$) with the 10-bit input. $(LUT \times \text{latency})^{-1}$ is counted to measure the area efficiency of each method. With 10-bit input data, the area efficiency of BN-SB as a multiplier is 20.89 times that of the classic method, 48.9 times that of the deterministic method, and 8.53 times that of the BISC method. The energy consumption of one multiplication for four methods is also tested at 400-MHz clock. The energy consumption of AFE SC reduces 95.3%, 97.2%, and 82.8% than classic method, deterministic method, and BISC method, respectively.

IV. CONCLUSION

A new encoding method of SC, AFE, was presented. The proposed method uses the expectation of multibit streams to achieve SC. Compared with the conventional SB, the AFE realizes low-latency and low-area occupation instead of the long latency of the classic method. The rationality of the computing rules for the AFE stream was proven, and the basic circuit for AFE SC was designed. For quadratic polynomial operations in particular, it only requires the use of the BN-SB circuits of AFE without additional multiplier circuits. In the experiments for precision, using BN-SB as a multiplier reduced the latency by a factor of $1/2^m$ to reach zero error calculation. With 10-bit input data, the area efficiency of BN-SB as a multiplier is 20.89 times that of the classic SC method.

REFERENCES

[1] B. R. Gaines, "Stochastic computing systems," in *Advances in Information Systems Science*, J. T. Tou, Ed. New York, NY, USA: Springer, 1969, pp. 37–172.

[2] Y. Liu, S. Liu, Y. Wang, F. Lombardi, and J. Han, "A survey of stochastic computing neural networks for machine learning applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 7, pp. 2809–2824, Jul. 2021.

[3] M. H. Najafi, S. Jamali-Zavareh, D. J. Lilja, M. D. Riedel, K. Bazargan, and R. Harjani, "Time-encoded values for highly efficient stochastic circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 5, pp. 1644–1657, May 2017.

[4] R. K. Budhwani, R. Ragavan, and O. Sentieys, "Taking advantage of correlation in stochastic computing," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4.

[5] P. Li, D. J. Lilja, W. Qian, K. Bazargan, and M. D. Riedel, "Computation on stochastic bit streams digital image processing case studies," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 3, pp. 449–462, Mar. 2014.

[6] S.-J. Min, E.-W. Lee, and S.-I. Chae, "A study on the stochastic computation using the ratio of one pulses and zero pulses," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, vol. 6, May/Jun. 1994, pp. 471–474.

[7] S. Sato, K. Nemoto, S. Akimoto, M. Kinjo, and K. Nakajima, "Implementation of a new neurochip using stochastic logic," *IEEE Trans. Neural Netw.*, vol. 14, no. 5, pp. 1122–1127, Sep. 2003.

[8] H. Li, Y. Hayakawa, S. Sato, and K. Nakajima, "Hardware implementation of an inverse function delayed neural network using stochastic logic," *IEICE Trans. Inf. Syst.*, vol. E89-D, no. 9, pp. 2572–2578, Sep. 2006.

[9] H. Sim, D. Nguyen, J. Lee, and K. Choi, "Scalable stochastic-computing accelerator for convolutional neural networks," in *Proc. 22nd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2017, pp. 696–701.

[10] H. Sim and J. Lee, "A new stochastic computing multiplier with application to deep convolutional neural networks," in *Proc. 54th Annu. Design Autom. Conf. (DAC)*, Jun. 2017, p. 29.

[11] K. Kim, J. Kim, J. Yu, J. Seo, J. Lee, and K. Choi, "Dynamic energy-accuracy trade-off using stochastic computing in deep neural networks," in *Proc. 53rd Annu. Design Autom. Conf. (DAC)*, Jun. 2016, p. 124.

[12] W. Qian, X. Li, M. D. Riedel, K. Bazargan, and D. J. Lilja, "An architecture for fault-tolerant computation with stochastic logic," *IEEE Trans. Comput.*, vol. 60, no. 1, pp. 93–105, Jan. 2011.

[13] B. Moons and M. Verhelst, "Energy-efficiency and accuracy of stochastic computing circuits in emerging technologies," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 4, no. 4, pp. 475–486, Dec. 2014.

[14] S. Liu and J. Han, "Toward energy-efficient stochastic circuits using parallel Sobol sequences," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 7, pp. 1326–1339, Jul. 2018.

[15] S. Liu and J. Han, "Energy efficient stochastic computing with Sobol sequences," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 650–653.

[16] P. Bratley and B. L. Fox, "Algorithm 659: Implementing Sobol's quasirandom sequence generator," *ACM Trans. Math. Softw.*, vol. 14, no. 1, pp. 88–100, Mar. 1988.

[17] D. Jenson and M. Riedel, "A deterministic approach to stochastic computation," in *Proc. 35th Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–8.

[18] M. H. Najafi, D. Jenson, D. J. Lilja, and M. D. Riedel, "Performing stochastic computation deterministically," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2925–2938, Dec. 2019.

[19] M. H. Najafi and D. J. Lilja, "High quality down-sampling for deterministic approaches to stochastic computing," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 1, pp. 7–14, Jan. 2021.

[20] K. K. Parhi, "Analysis of stochastic logic circuits in unipolar, bipolar and hybrid formats," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4.

[21] V. T. Lee, A. Alaghi, J. P. Hayes, V. Sathe, and L. Ceze, "Energy-efficient hybrid stochastic-binary neural networks for near-sensor computing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 13–18.

[22] G. Casella and R. L. Berger, "Transformations and expectations," in *Statistical Inference*, vol. 2, 2nd ed. Belmont, CA, USA: Wadsworth, 2002, pp. 47–55.

[23] A. Alaghi and J. P. Hayes, "Fast and accurate computation using stochastic circuits," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2014, pp. 1–4, doi: [10.7873/DATE.2014.089](https://doi.org/10.7873/DATE.2014.089).

[24] M. H. Najafi, D. J. Lilja, and M. Riedel, "Deterministic methods for stochastic computing using low-discrepancy sequences," in *Proc. Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2018, pp. 1–8.